The Journal of Open Source Software

adaptivelasso: A JAX-based Python Library for Adaptive and Structured Sparse Regression

Linpei Zhang[1*], **Jui-Ting Chang**[1*], **and Nathan Wycoff**[2*]

1 Independent Researcher **2** Department of Mathematics and Statistics, University of Massachusetts, Amherst * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 30 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

In data-rich research settings, predictive accuracy alone is often insufficient: researchers also need to understand which variables contribute to a model. Variable selection addresses this need by identifying a smaller set of predictors from a large collection, among them sparsity-inducing penalties, such as the Lasso (Tibshirani, 1996), which perform selection as part of model fitting. Researchers may also have information about relationships among predictors. For example, several measurements may describe a common process and be expected to enter a model together (Yuan & Lin, 2006), or an interaction may be more interpretable when its corresponding main effects are also selected (Bien et al., 2013). Incorporating these relationships becomes more challenging when responses are counts, observations are heavy-tailed, or interaction terms greatly enlarge the feature space.

Specialized solvers can exploit a particular model and sparsity structure efficiently, but exploring different combinations can require substantial implementation work. Wycoff et al. (2024) develop a proximal optimization framework that jointly estimates regression coefficients and their penalty weights, with priors on those weights encouraging the desired sparsity structure. This provides a common statistical foundation for combining observation models and structural assumptions without deriving a separate structured proximal operator for each prior, but the accompanying code is research-grade, and not optimized for fitting into existing workflows.

`adaptivelasso` develops this framework into a Python library for fitting and comparing sparse regression models. Implemented with JAX (Bradbury et al., 2018), it provides Gaussian, Bernoulli, Poisson, Negative Binomial, Cauchy, and zero-inflated Negative Binomial likelihoods alongside adaptive, group, and hierarchical penalty priors. Its software contributions include coordinating variance-reduced minibatch optimization with on-demand interaction features and introducing likelihood-aware regularization search. A scikit-learn-compatible interface (Pedregosa et al., 2011) connects the fitting engine to familiar preprocessing and model-selection workflows, helping researchers explore alternative model specifications within a single computational framework.

Statement of Need

Sparse regression is useful when prediction must be accompanied by a manageable set of explanatory variables. The Lasso (Tibshirani, 1996) performs selection through a common penalty, while the Adaptive Lasso (Zou, 2006) allows different amounts of shrinkage for different coefficients. Applied workflows often also need to accommodate overdispersed counts, outliers, or related predictors. Groups and interactions introduce further choices about which variables should be encouraged to enter a model together (Bien et al., 2013; Yuan & Lin, 2006).

Combining these requirements creates a software problem: changing a likelihood or sparsity

DRAFT

structure can require a different solver, tuning scale, feature representation, and evaluation procedure. `adaptivelasso` targets researchers comparing such combinations within one analysis, and methods researchers extending the underlying objective. Its built-in Gaussian, Bernoulli, Poisson, Negative Binomial, Cauchy, and zero-inflated Negative Binomial likelihoods share independent adaptive, group, and hierarchical penalty components. Structure is encouraged through priors on penalty weights; it does not impose hard group-selection or interaction-heredity constraints.

State of the Field

`adaptivelasso` supports a variety of likelihood functions and prior structures. We compare it with established sparse regression tools in the Python and R ecosystems: scikit-learn's `LassoCV` (Pedregosa et al., 2011); R's `glmnet` (Friedman et al., 2010), the standard coordinate-descent solver for penalized GLMs; `ncvreg` (Breheny & Huang, 2011), which targets the same estimation-bias problem as the Adaptive Lasso but through nonconvex (MCP/SCAD) penalties rather than jointly-learned weights; its group-selection counterpart `grpreg` (Breheny & Huang, 2009); and `pyglmnet` (Jas et al., 2020), a pure-Python elastic-net GLM library.

Joint estimation requires the optimizer to maintain and update both regression coefficients and penalty weights, including their coupled proximal step, which requires different control structure than traditional penalties. A dedicated fitting engine supports that joint state and its structural priors, while the scikit-learn interface reuses established preprocessing and model-selection tools.

The table distinguishes jointly learned penalty weights and soft priors on those weights from fixed penalty factors and group-selection penalties. The scikit-learn column refers specifically to `LassoCV`; related estimators are identified in the notes.

Feature	`adaptivelasso`	`LassoCV`	`glmnet`	`ncvreg`	`grpreg`	`pyglmnet`
Gaussian	Yes	Yes	Yes	Yes	Yes	Yes
Bernoulli	Yes	No[1]	Yes	Yes	Yes	Yes
Poisson	Yes	No[2]	Yes	Yes	Yes	Yes
Negative Binomial	Yes	No	Yes[6]	No	No	No
Cauchy	Yes	No	No	No	No	No
Zero-inflated NB	Yes	No	No	No	No	No
Learned penalty weights	Yes	No	No[3]	No[4]	No[4]	No
Soft structured priors	Yes	No	No	No	No[5]	No[5]

[1] *Requires `LogisticRegression(penalty='l1')`.* [2] *Requires `PoissonRegressor` (no built-in CV path).* [3] *`glmnet` accepts a user-supplied, fixed `penalty.factor` per coefficient (enabling a manual two-step adaptive lasso), but does not jointly learn it with β.* [4] *Debiasing is instead achieved via a nonconvex penalty shape (MCP/SCAD), a fixed-form alternative to jointly learning a per-coefficient weight.* [5] *`grpreg` supports group-selection and bi-level penalties, allowing selection both between and within groups; `pyglmnet` supports group lasso. These penalties differ from the soft group/hierarchical priors on learned penalty weights used here (see the grpreg documentation and pyglmnet documentation).*

[6] *`glmnet` supports Negative Binomial regression through a family object such as `MASS::negative.binomial(theta = ...)`, with dispersion specified by the user. In `adaptivelasso`, dispersion is estimated during fitting (see the glmnet family documentation).*

DRAFT

The Journal of Open Source Software

Software Design

`adaptivelasso` is designed around a recurring research task, where the response distribution and sparsity assumptions vary. Its software contribution is an integrated fitting engine that coordinates model components, expanded feature representations, and regularization search. The architecture separates likelihood and prior specification from optimization, using JAX differentiation for smooth terms and the joint coefficient-and-penalty proximal update of Wycoff et al. (2024) for the nonsmooth penalty to facilitate applications of the software. Custom likelihood-gradient and prior interfaces provide extension points for methods researchers. This separation allows new model specifications to reuse the optimizer and feature-handling infrastructure, reducing the implementation work needed to explore alternative sparse models.

A central engineering contribution is coordinating interaction-feature construction with variance-reduced optimization. Expanding all observations in advance can make interaction models expensive to store, even when the original data fit comfortably in memory. We integrated on-demand expansion with both SVRG minibatch updates and chunked full-data reference gradients, and applied the corresponding transformation during prediction. Researchers can therefore supply the original predictors throughout the workflow while fitting models with interaction or quadratic terms. Repeated feature construction trades computation for a smaller expanded training-matrix workspace, a design feature that allows out-of-memory computation for large datasets.

A second contribution addresses the practical cost of moving between likelihoods. Since useful penalty scales can differ substantially, we developed a regularization search based on coefficient gradients at an intercept-only initialization, with likelihood-specific scale or dispersion estimates. In the example workflow, the selected relative penalty is rescaled before refitting on the full training set. Scikit-learn-compatible estimator interfaces connect the fitting engine to existing preprocessing and model-selection tools (Pedregosa et al., 2011), allowing researchers to reuse familiar analysis components alongside the package's specialized optimization.

Research Impact Statement

The code for Wycoff et al. (2024) has been refactored to this new standard, and we show just one example here. This repository includes an end-to-end analysis of hourly casual rentals in the UCI Bike Sharing dataset (Fanaee-T & Gama, 2014). This count response illustrates why likelihood choice should be available within a common sparse-regression workflow. The experiment scripts provide seeded train/validation/test splits, training-only preprocessing, likelihood-specific regularization scaling, validation-based selection, and refitting before test evaluation.

The following results illustrate that workflow with interaction features. RMSE measures point-prediction error; the continuous ranked probability score (CRPS) assesses predictive distributions, with lower values preferred for both metrics. For `LassoCV`, the experiment constructs a Gaussian predictive distribution using the training residual scale.

Model	Likelihood	RMSE	CRPS
`LassoCV`	Gaussian	19.86	10.05
`adaptivelasso`	Gaussian	19.84	10.06
`adaptivelasso`	Poisson	15.90	7.16
`adaptivelasso`	Negative Binomial	16.46	6.64

The Gaussian rows have similar scores, while the count-model rows demonstrate the practical value of changing the observation model within the same software. This single dataset does not establish a general advantage of adaptive penalties or separate their effect from likelihood choice.

DRAFT

The workflow is implemented in `experiments/hourly_compare.py`, with shared preprocessing and evaluation in `experiments/hourly_common.py`; more methodological comparisons are given in Wycoff et al. (2024).

AI Usage Disclosure

We used AI tools, including Claude Code with Opus 5.5, for code drafting, refactoring, and drafting the report. We made the core design decisions, and checked and edited the code and text afterwards for accuracy and take full responsibility for correctness of the resulting products.

Acknowledgements

NW gratefully acknowledges funding from the Massive Data Institute, the Tech and Public Policy Grant Program, and NSF/NGA grant #2428033. The computational resources for this work were provided by the UMass Amherst's partnership with the Unity Research Computing Platform, a multi-institutional cluster led by the University of Massachusetts and the University of Rhode Island.

References

Bien, J., Taylor, J., & Tibshirani, R. (2013). A lasso for hierarchical interactions. *The Annals of Statistics*, *41*(3), 1111–1141. https://doi.org/10.1214/13-AOS1096

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Katariya, Y., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs*. https://github.com/jax-ml/jax

Breheny, P., & Huang, J. (2009). Penalized methods for bi-level variable selection. *Statistics and Its Interface*, *2*(3), 369–380. https://doi.org/10.4310/sii.2009.v2.n3.a10

Breheny, P., & Huang, J. (2011). Coordinate descent algorithms for nonconvex penalized regression, with applications to biological feature selection. *The Annals of Applied Statistics*, *5*(1), 232–253. https://doi.org/10.1214/10-AOAS388

Fanaee-T, H., & Gama, J. (2014). Event labeling combining ensemble detectors and background knowledge. *Progress in Artificial Intelligence*, *2*, 113–127. https://doi.org/10.1007/s13748-013-0040-3

Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, *33*(1), 1–22. https://doi.org/10.18637/jss.v033.i01

Jas, M., Achakulvisut, T., Idrizović, A., Acuna, D., Antalek, M., Marques, V., Odland, T., Garg, R. P., Agrawal, M., Umegaki, Y., Foley, P., Fernandes, H., Harris, D., Li, B., Pieters, O., Otterson, S., De Toni, G., Rodgers, C., Dyer, E., … Ramkumar, P. (2020). Pyglmnet: Python implementation of elastic-net regularized generalized linear models. *Journal of Open Source Software*, *5*(47), 1959. https://doi.org/10.21105/joss.01959

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*(85), 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, *58*(1), 267–288. https://doi.org/10.1111/j.

DRAFT

2517-6161.1996.tb02080.x

Wycoff, N., Singh, L. O., Arab, A., & Donato, K. M. (2024). Proximal iteration for nonlinear adaptive lasso. *arXiv Preprint arXiv:2412.05726*. https://doi.org/10.48550/arXiv.2412.05726

Yuan, M., & Lin, Y. (2006). Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, *68*(1), 49–67. https://doi.org/10.1111/j.1467-9868.2005.00532.x

Zou, H. (2006). The adaptive lasso and its oracle properties. *Journal of the American Statistical Association*, *101*(476), 1418–1429. https://doi.org/10.1198/016214506000000735

DRAFT